Machine Learning - Tutorial

Convex Optimization, and Perceptron Learning Algorithm

Teaching Assistant: Sourav Goswami

March 14, 2026

Overview

A. Convex optimization – example problem and its solution

B. Perceptron Learning Algorithm

   1. Hyperplanes, Linearly separable data
   2. Perceptron
   3. Algorithm, Geometric intuition
   4. Evolution of decision boundary
   5. Coding example
   6. Review of concepts

Convex Optimization

Example problem

Minimize $f = x_1^2 + x_2^2 + 60x_1$

subject to $h_1 = x_1 - 80 \geq 0$

$h_2 = x_1 + x_2 - 120 \geq 0$

Solution

Minimize $f = x_1^2 + x_2^2 + 60x_1$

subject to $h_1 = x_1 - 80 \geq 0$

$h_2 = x_1 + x_2 - 120 \geq 0$

Primal problem:

$$\min_x \quad f(x)$$
$$\text{s.t.} \quad g_i(x) \leq 0 \quad i = 1, \ldots, m$$

Step 0: Re-write the problem in standard form

$f = x_1^2 + x_2^2 + 60x_1$

$g_1 = -x_1 + 80 \leq 0$

$g_2 = -x_1 - x_2 + 120 \leq 0$

Convexity check?

Hessian matrix is positive definite!

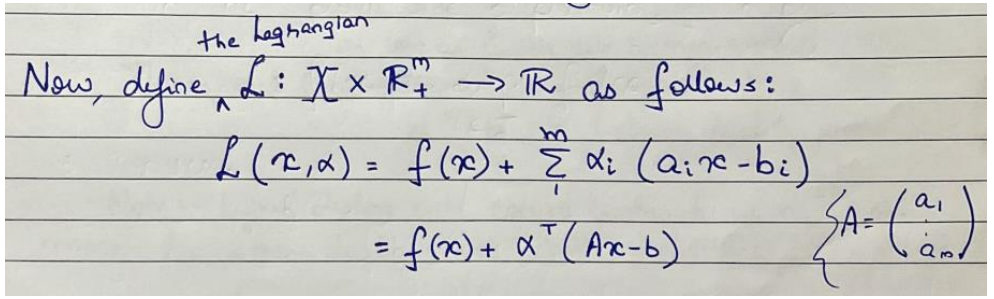
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Solution

Step 1: Construct the Lagrangian

$$L(x, \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x)$$

$$L(x_1, x_2, \lambda_1, \lambda_2) = x_1^2 + x_2^2 + 60x_1 + \lambda_1(80 - x_1) + \lambda_2(120 - x_1 - x_2)$$



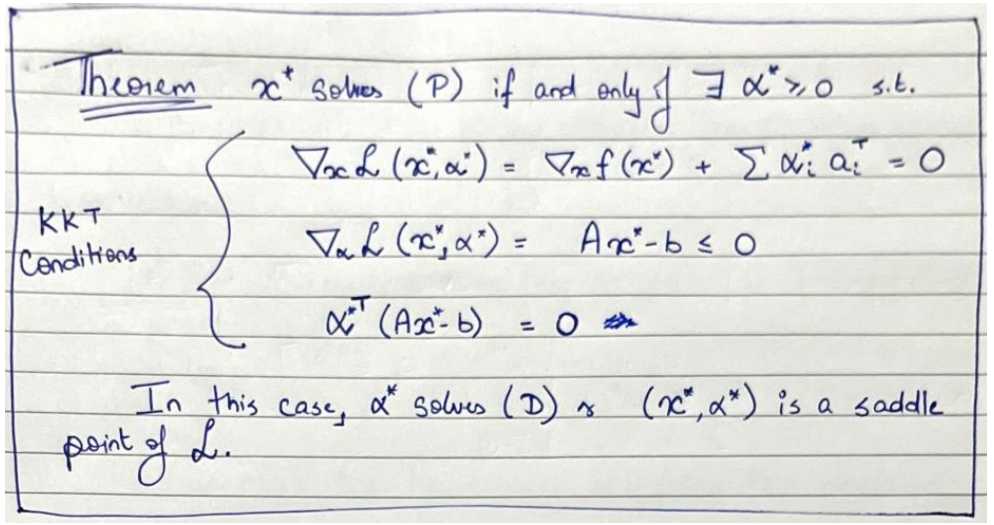
Step 2: List the KKT conditions

$$\frac{\partial f}{\partial x_i} + \sum_{j=1}^{m} \lambda_j \frac{\partial g}{\partial x_i} = 0 \qquad i = 1, 2, \ldots, n$$

$$\lambda_j g_j = 0 \qquad j = 1, 2, \ldots, m$$

$$g_j \le 0 \qquad j = 1, 2, \ldots, m$$

$$\lambda_j \ge 0 \qquad j = 1, 2, \ldots, m$$



Solution

Step 2: List the KKT conditions

$$\frac{\partial L}{\partial x_1} = 2x_1 + 60 - \lambda_1 - \lambda_2 = 0$$

$$\frac{\partial L}{\partial x_2} = 2x_2 - \lambda_2 = 0$$

$$\lambda_1(80 - x_1) = 0$$

$$\lambda_2(120 - x_1 - x_2) = 0$$

$$80 - x_1 \leq 0$$

$$120 - x_1 - x_2 \leq 0$$

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

$$\frac{\partial f}{\partial x_i} + \sum_{j=1}^{m} \lambda_j \frac{\partial g}{\partial x_i} = 0 \qquad i = 1, 2, \ldots, n$$

$$\lambda_j g_j = 0 \qquad j = 1, 2, \ldots, m$$

$$g_j \leq 0 \qquad j = 1, 2, \ldots, m$$

$$\lambda_j \geq 0 \qquad j = 1, 2, \ldots, m$$

Solution

Step 3: Applying KKT conditions to find the solution

$$\frac{\partial L}{\partial x_1} = 2x_1 + 60 - \lambda_1 - \lambda_2 = 0$$

$$\frac{\partial L}{\partial x_2} = 2x_2 - \lambda_2 = 0$$

$$x_2 = \frac{\lambda_2}{2}$$

$$x_1 = \frac{\lambda_1 + \lambda_2 - 60}{2}$$

$$\lambda_1(80 - x_1) = 0$$

$$\lambda_2(120 - x_1 - x_2) = 0$$

Test Case A: Both constraints are INACTIVE ($\lambda_1 = 0, \lambda_2 = 0$)

If multipliers are 0, then $x_2 = 0$ and $x_1 = -30$.

Check Primal Feasibility: Does $80 - (-30) \leq 0$?

$$80 - x_1 \leq 0$$

$$120 - x_1 - x_2 \leq 0$$

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

Solution

Step 3: Applying KKT conditions to find the solution

$$\frac{\partial L}{\partial x_1} = 2x_1 + 60 - \lambda_1 - \lambda_2 = 0$$

$$\frac{\partial L}{\partial x_2} = 2x_2 - \lambda_2 = 0$$

$$x_2 = \frac{\lambda_2}{2}$$

$$x_1 = \frac{\lambda_1 + \lambda_2 - 60}{2}$$

$$\lambda_1(80 - x_1) = 0$$

$$\lambda_2(120 - x_1 - x_2) = 0$$

Test Case B: Constraint 1 is Inactive, Constraint 2 is Active ($\lambda_1 = 0, \lambda_2 > 0$)

$$x_1 = \frac{0 + \lambda_2 - 60}{2} = \frac{\lambda_2 - 60}{2} \qquad x_2 = \frac{\lambda_2}{2}$$

$$80 - x_1 \leq 0$$

$$120 - x_1 - x_2 \leq 0$$

$$120 - x_1 - x_2 = 0 \implies \lambda_2 = 150$$

$$80 - x_1 \leq 0$$

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

Solution

Step 3: Applying KKT conditions to find the solution

$$\frac{\partial L}{\partial x_1} = 2x_1 + 60 - \lambda_1 - \lambda_2 = 0$$

$$\frac{\partial L}{\partial x_2} = 2x_2 - \lambda_2 = 0$$

$$x_2 = \frac{\lambda_2}{2}$$

$$x_1 = \frac{\lambda_1 + \lambda_2 - 60}{2}$$

$$\lambda_1(80 - x_1) = 0$$

$$\lambda_2(120 - x_1 - x_2) = 0$$

Test Case C: Constraint 1 is Active, Constraint 2 is Inactive ($\lambda_1 > 0, \lambda_2 = 0$)

$$80 - x_1 = 0 \implies x_1 = 80$$

$$x_2 = \frac{\lambda_2}{2} = \frac{0}{2} \implies x_2 = 0$$

$$80 - x_1 \leq 0$$

$$120 - x_1 - x_2 \leq 0$$

$$120 - x_1 - x_2 \leq 0$$

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

Solution

Step 3: Applying KKT conditions to find the solution

$$\frac{\partial L}{\partial x_1} = 2x_1 + 60 - \lambda_1 - \lambda_2 = 0$$

$$\frac{\partial L}{\partial x_2} = 2x_2 - \lambda_2 = 0$$

$$2(40) - \lambda_2 = 0 \implies \lambda_2 = 80 \text{ (Valid, it is } \geq 0)$$

$$2(80) + 60 - \lambda_1 - 80 = 0 \implies 220 - \lambda_1 = 80 \implies \lambda_1 = 140 \text{ (Valid, it is } \geq 0)$$

$$\lambda_1(80 - x_1) = 0$$

$$\lambda_2(120 - x_1 - x_2) = 0$$

Both constraints are ACTIVE ($\lambda_1 > 0, \lambda_2 > 0$)

$$80 - x_1 = 0 \implies x_1 = 80$$

$$120 - x_1 - x_2 = 0 \implies 120 - 80 - x_2 = 0 \implies x_2 = 40$$

$$80 - x_1 \leq 0$$

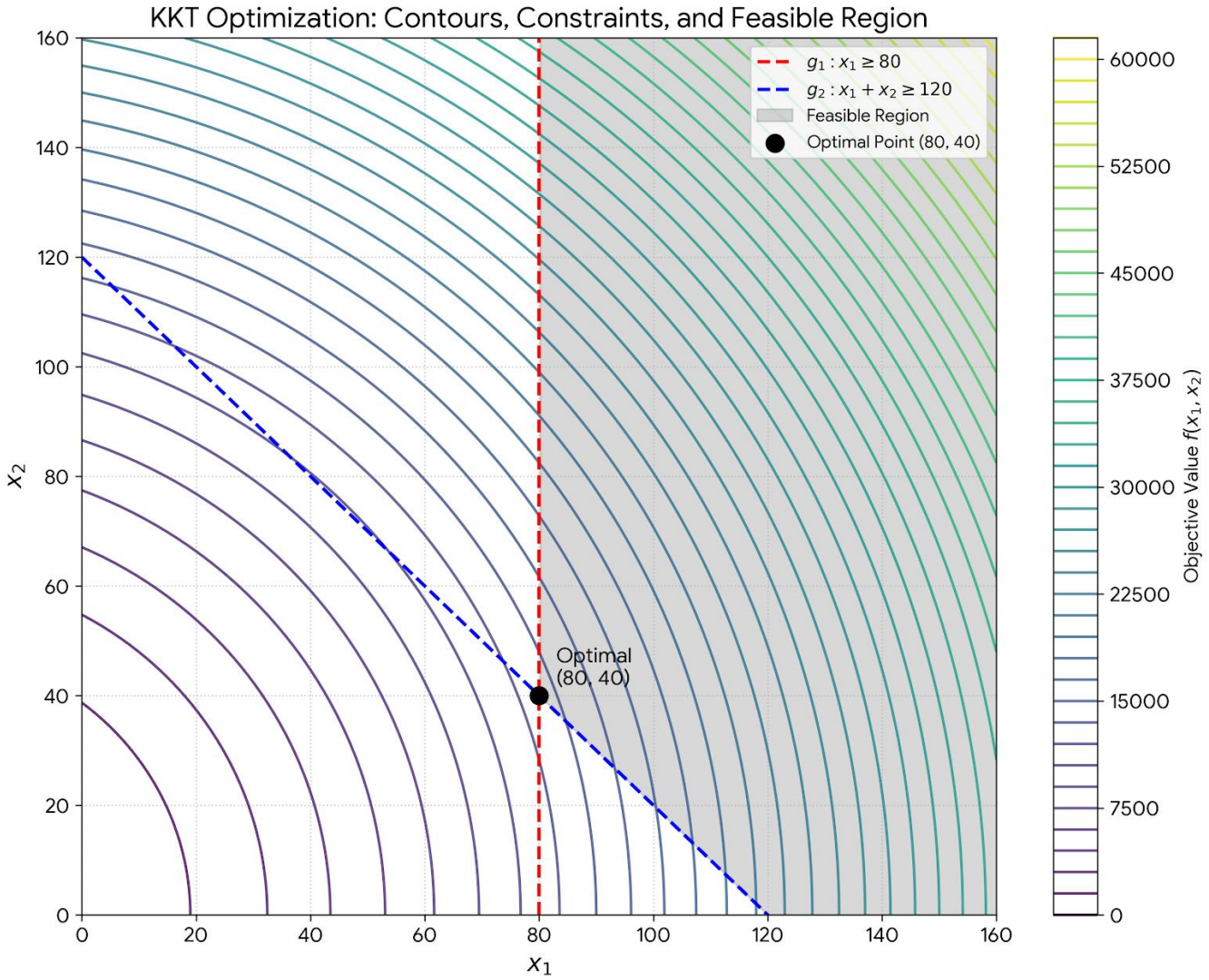
$$120 - x_1 - x_2 \leq 0$$

The Primal Solution: $x_1^* = 80, x_2^* = 40.$

The Optimal Objective Value: $f(80, 40) = 80^2 + 40^2 + 60(80) = 6400 + 1600 + 4800 = \mathbf{12800}.$

$$\lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

Visualizing the solution



Perceptron Learning Algorithm

Hyperplane

In a p-dimensional space, a *hyperplane* is a flat affine subspace of dimension $p - 1$.[1] For instance, in two dimensions, a hyperplane is a flat one-dimensional subspace—in other words, a line. In three dimensions, a hyperplane is a flat two-dimensional subspace—that is, a plane. In $p > 3$ dimensions, it can be hard to visualize a hyperplane, but the notion of a $(p - 1)$-dimensional flat subspace still applies.

The mathematical definition of a hyperplane is quite simple. In two dimensions, a hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0 \tag{9.1}$$

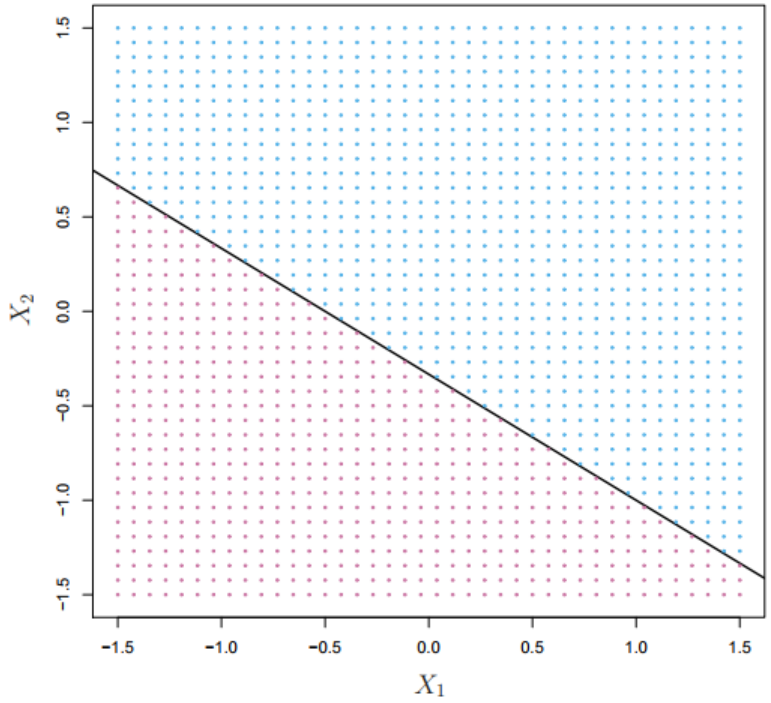
for parameters β_0, β_1, and β_2. When we say that (9.1) "defines" the hyperplane, we mean that any $X = (X_1, X_2)^T$ for which (9.1) holds is a point on the hyperplane. Note that (9.1) is simply the equation of a line, since indeed in two dimensions a hyperplane is a line.

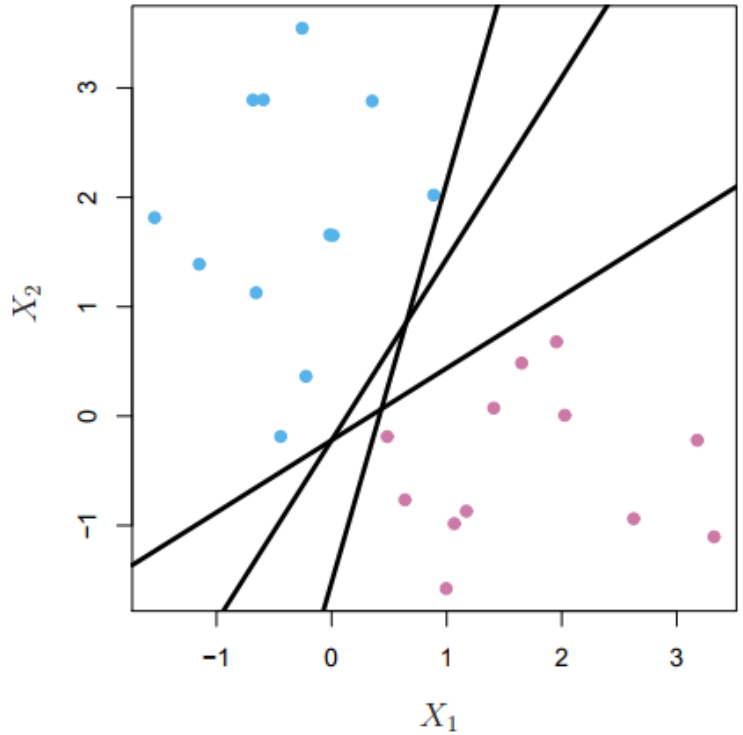
The word *affine* indicates that the subspace need not pass through the origin.

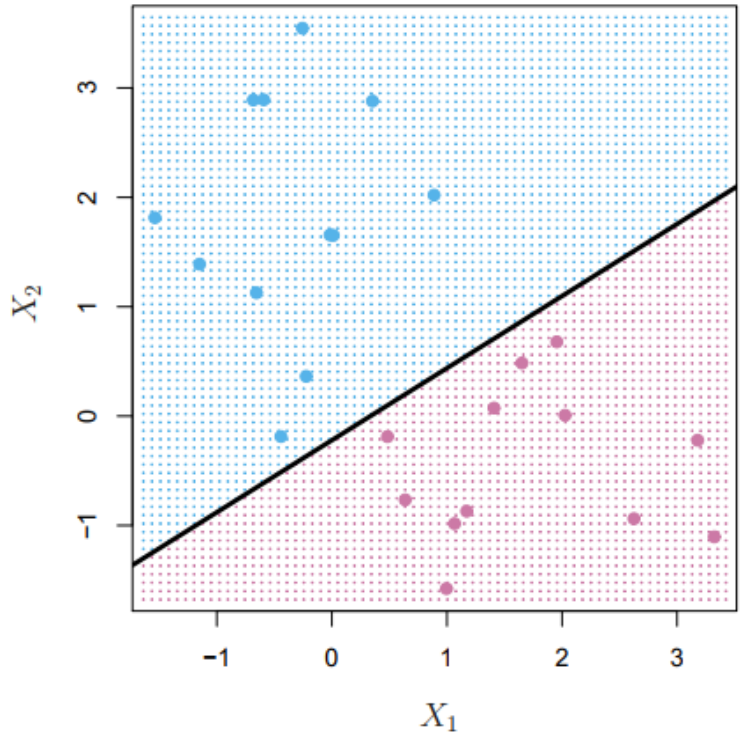
G. James et al., An Introduction to Statistical Learning, Springer Texts in Statistics, https://doi.org/10.1007/978-3-031-38747-0_9

Linearly separable data

In Euclidean geometry, **linear separability** is a property of two sets of points. This is most easily visualized in two dimensions (the Euclidean plane) by thinking of one set of points as being colored blue and the other set of points as being colored red. These two sets are *linearly separable* if there exists at least one line in the plane with all of the blue points on one side of the line and all the red points on the other side. This idea immediately generalizes to higher-dimensional Euclidean spaces if the line is replaced by a hyperplane.



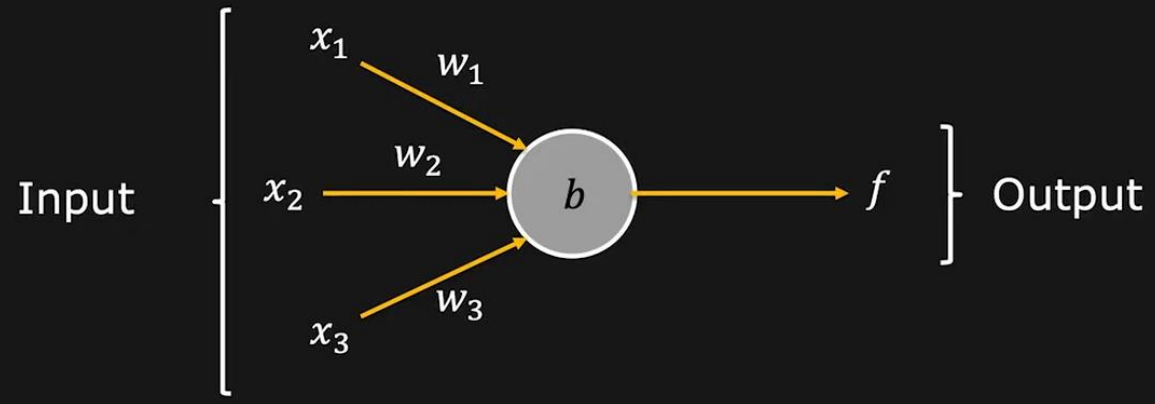




G. James et al., An Introduction to Statistical Learning, Springer Texts in Statistics, https://doi.org/10.1007/978-3-031-38747-0_9

Perceptron

Takes several inputs and gives a single binary output (Decision)



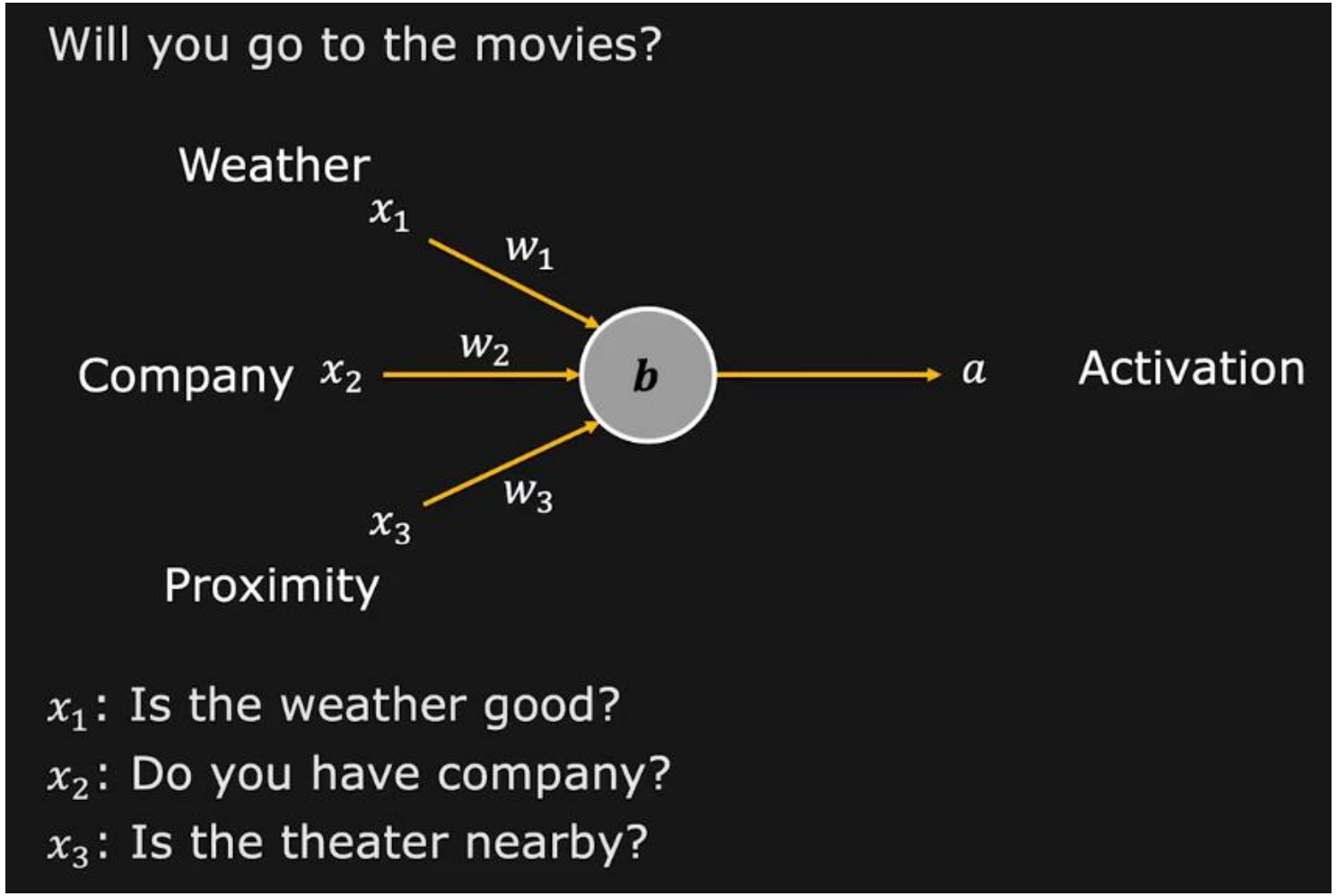
$$f = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq -b \\ 1 & \text{if } \sum_j w_j x_j > -b \end{cases} \qquad f = \begin{cases} 0 & \text{if } \boldsymbol{w}.\boldsymbol{x} + b \leq 0 \\ 1 & \text{if } \boldsymbol{w}.\boldsymbol{x} + b > 0 \end{cases}$$

where, $w_j = weights$

$b = bias\ (threshold)$

Prof. Shree K. Nayar (https://www.youtube.com/watch?v=OFbnpY_k7js)

Perceptron – Example



Assume binary inputs (0 or 1)

Go to the movies only if

1. the weather is good

AND

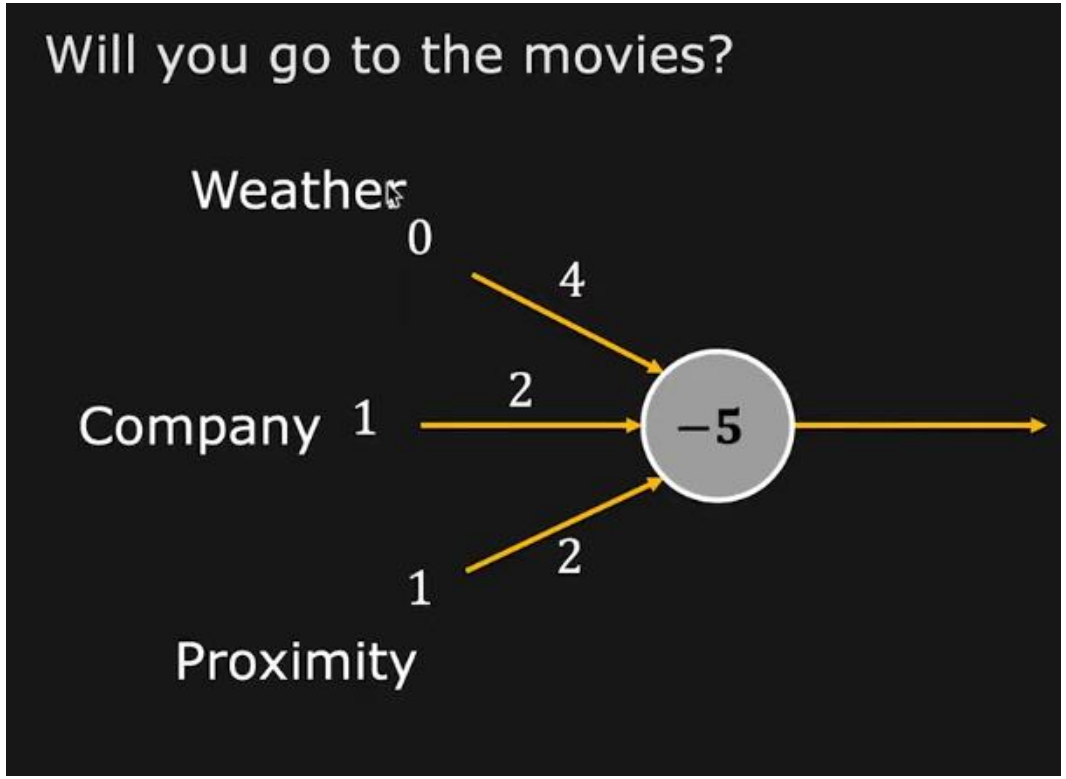
2. At least one other factor is favorable

What should be w_i?

What should be b?

Prof. Shree K. Nayar (https://www.youtube.com/watch?v=OFbnpY_k7js)

Perceptron – Example

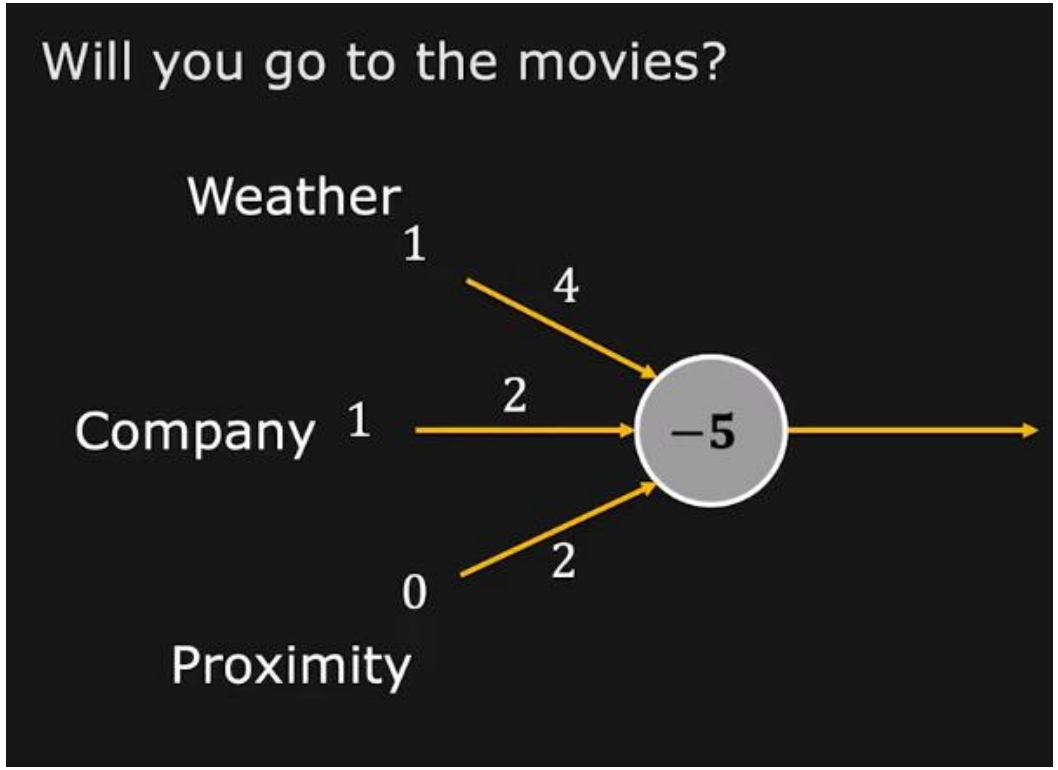


?

Scenario 1:

What is the decision for the given inputs and weights?

Perceptron – Example



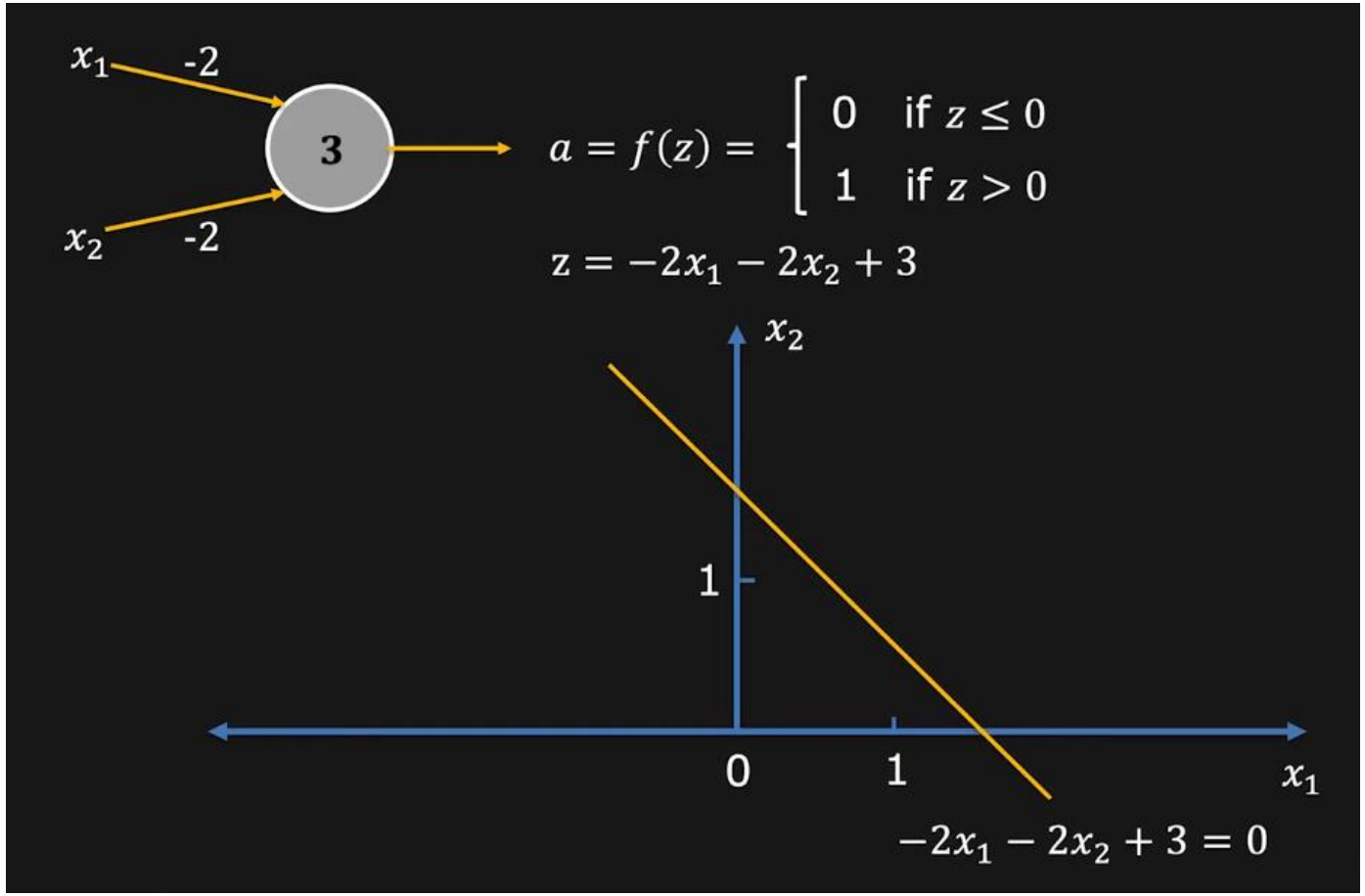
?

Scenario 2:

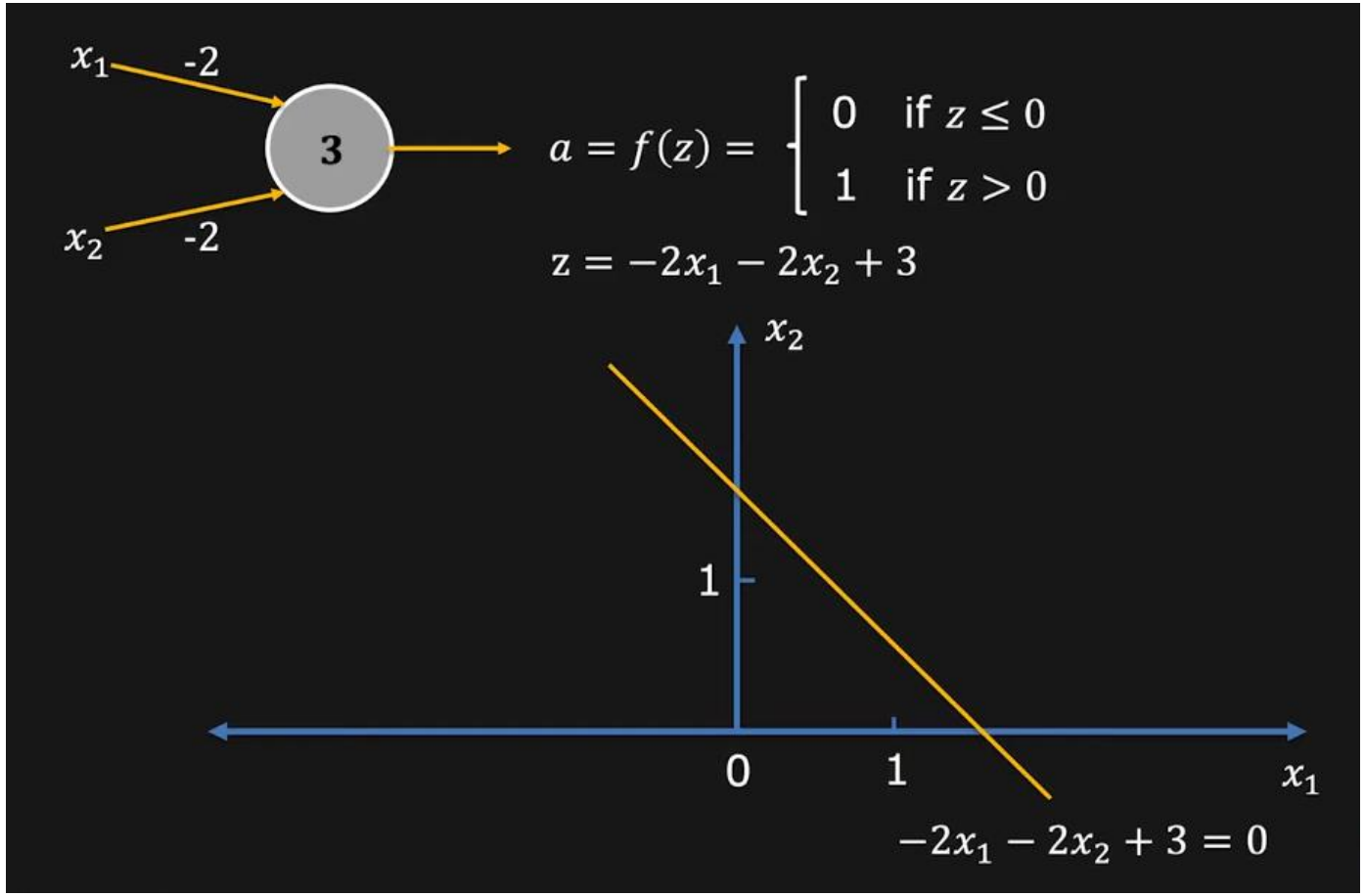
What is the decision for the given inputs and weights?

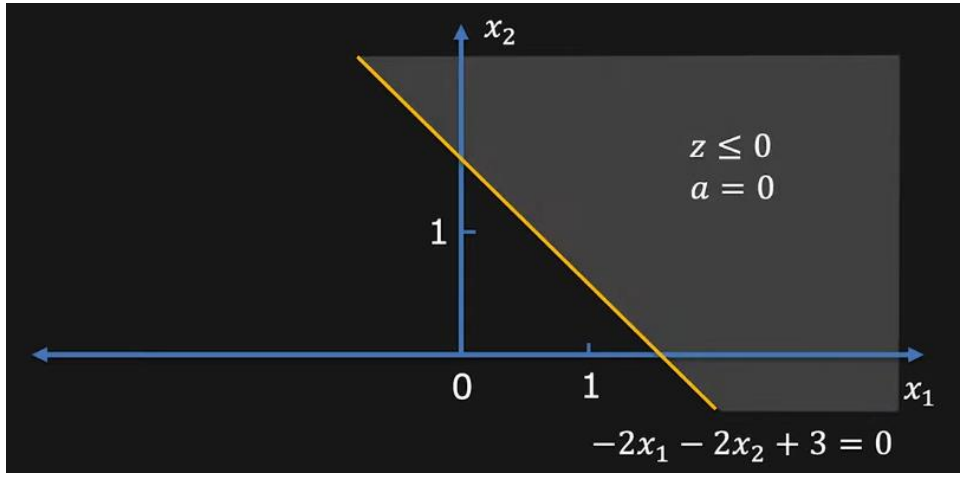
Prof. Shree K. Nayar (https://www.youtube.com/watch?v=OFbnpY_k7js)

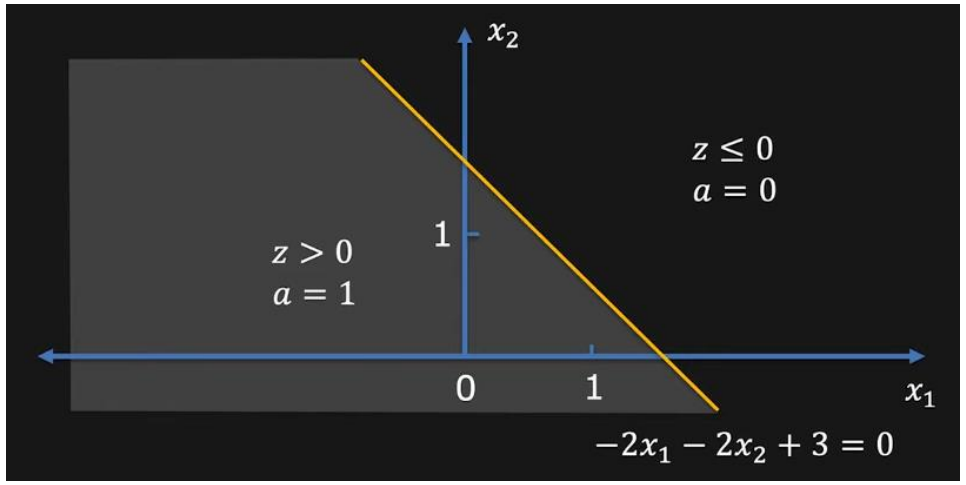
Perceptron as a Linear Classifier



Perceptron as a Linear Classifier





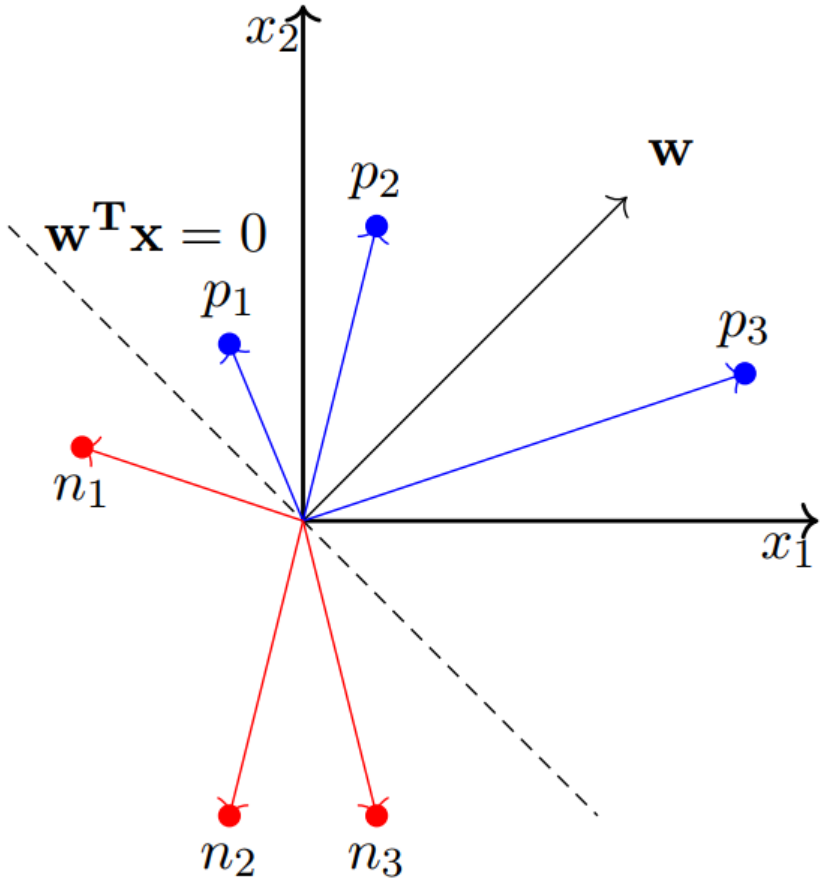


Perceptron Learning Algorithm (PLA)

Algorithm: Perceptron Learning Algorithm

$P \leftarrow inputs \quad with \quad label \quad 1;$
$N \leftarrow inputs \quad with \quad label \quad 0;$
Initialize **w** randomly;
while !*convergence* **do**
  Pick random $\mathbf{x} \in P \cup N$;
  if $\mathbf{x} \in P \quad and \quad \sum_{i=0}^{n} w_i * x_i < 0$ **then**
    $\mathbf{w} = \mathbf{w} + \mathbf{x}$;
  end
  if $\mathbf{x} \in N \quad and \quad \sum_{i=0}^{n} w_i * x_i \geq 0$ **then**
    $\mathbf{w} = \mathbf{w} - \mathbf{x}$;
  end
end
//the algorithm converges when all the inputs are classified correctly

PLA – Geometric intuition

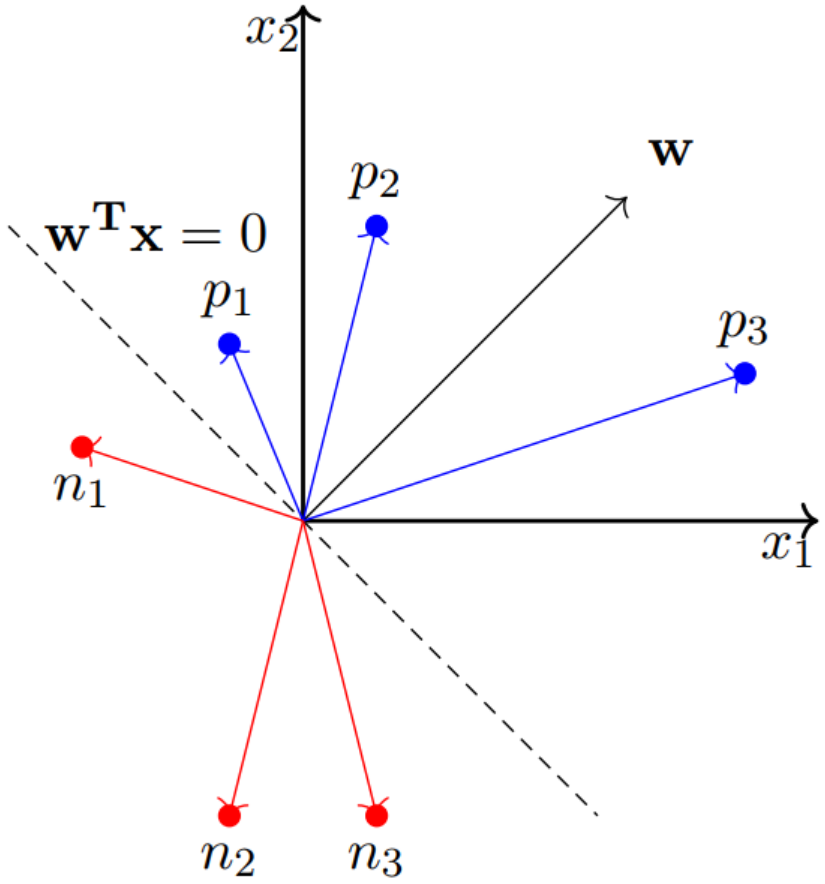


- For $\mathbf{x} \in P$ if $\mathbf{w}.\mathbf{x} < 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is greater than 90° (but we want α to be less than 90°)
- What happens to the new angle (α_{new}) when $\mathbf{w_{new}} = \mathbf{w} + \mathbf{x}$

$$
\begin{aligned}
cos(\alpha_{new}) &\propto \mathbf{w_{new}}^T \mathbf{x} \\
&\propto (\mathbf{w} + \mathbf{x})^T \mathbf{x} \\
&\propto \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x} \\
&\propto cos\alpha + \mathbf{x}^T \mathbf{x} \\
cos(\alpha_{new}) &> cos\alpha
\end{aligned}
$$

- Thus α_{new} will be less than α and this is exactly what we want

PLA – Geometric intuition



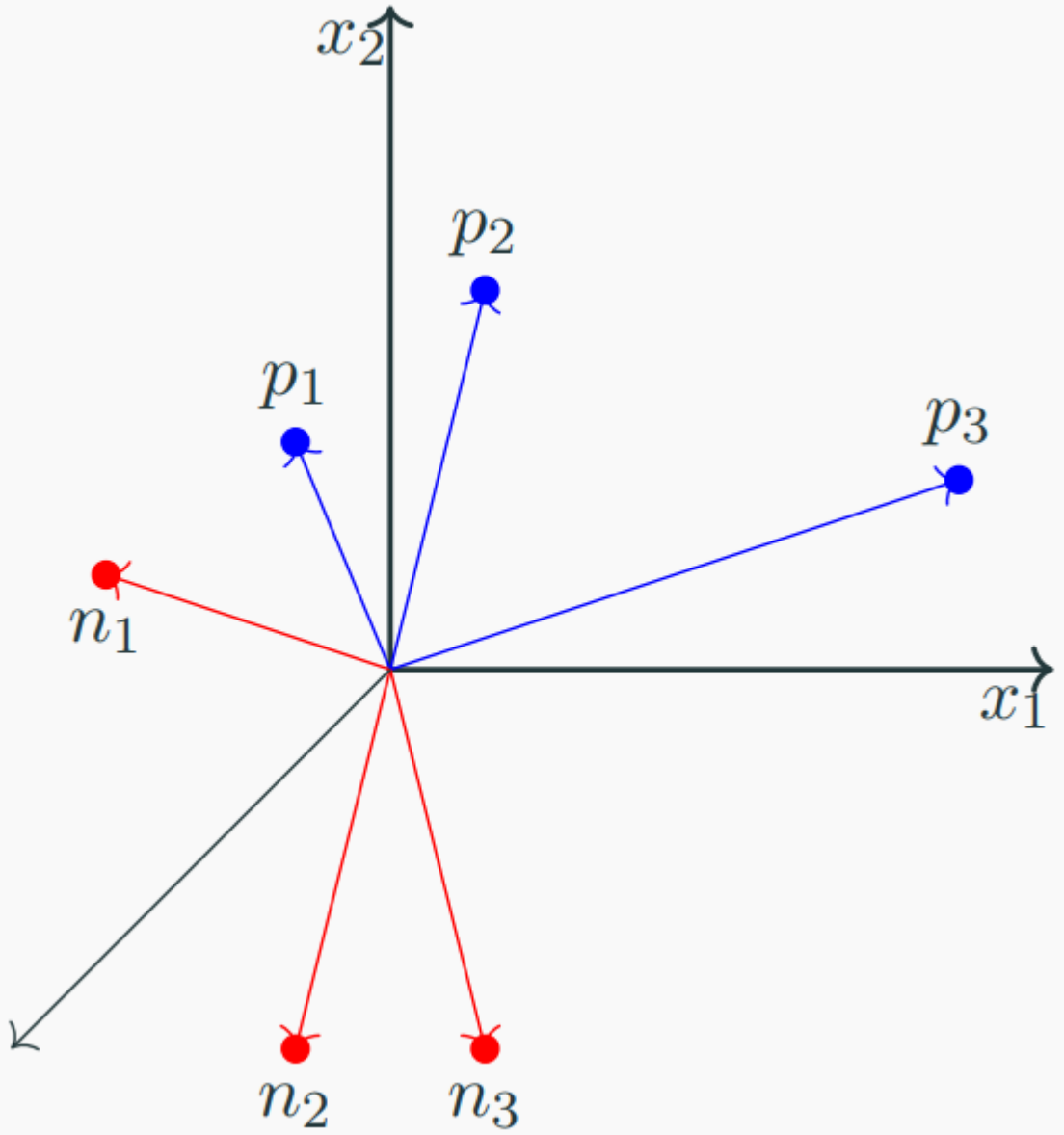
- For $\mathbf{x} \in N$ if $\mathbf{w}.\mathbf{x} \geq 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is less than $90°$ (but we want α to be greater than $90°$)
- What happens to the new angle (α_{new}) when $\mathbf{w_{new}} = \mathbf{w} - \mathbf{x}$

$$
\begin{aligned}
cos(\alpha_{new}) &\propto \mathbf{w_{new}}^T \mathbf{x} \\
&\propto (\mathbf{w} - \mathbf{x})^T \mathbf{x} \\
&\propto \mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x} \\
&\propto cos\alpha - \mathbf{x}^T \mathbf{x} \\
cos(\alpha_{new}) &< cos\alpha
\end{aligned}
$$

- Thus α_{new} will be greater than α and this is exactly what we want

Evolution of decision boundary

Decision boundary – Initial state

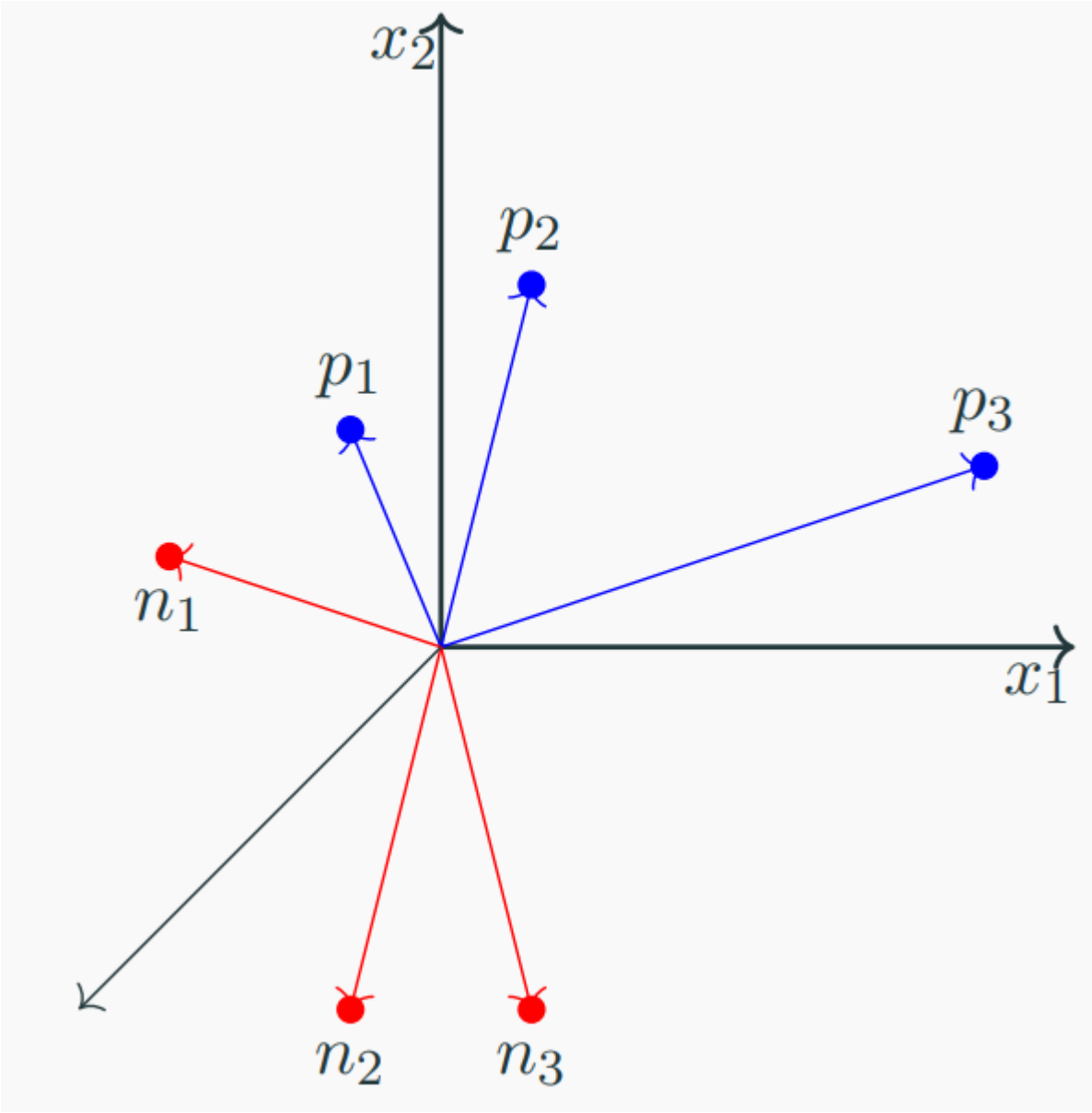


We initialized $\mathbf{w}$ to a random value

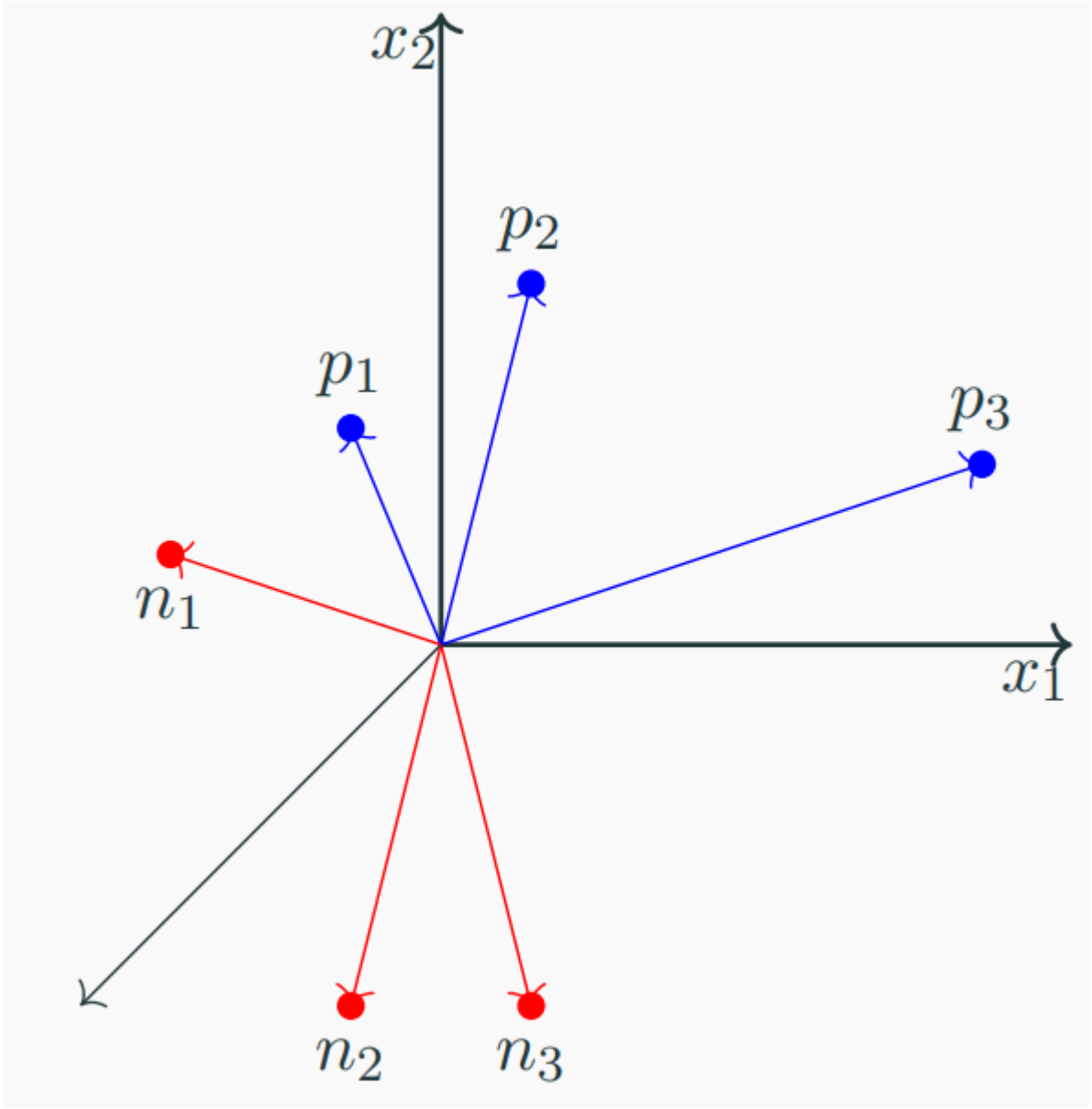
We observe that currently, $\mathbf{w} \cdot \mathbf{x} < 0$ ($\because$ angle $>$ 90°) for all the positive points and $\mathbf{w} \cdot \mathbf{x} \geq 0$ ($\because$ angle $<$ 90°) for all the negative points (the situation is exactly oppsite of what we actually want it to be)

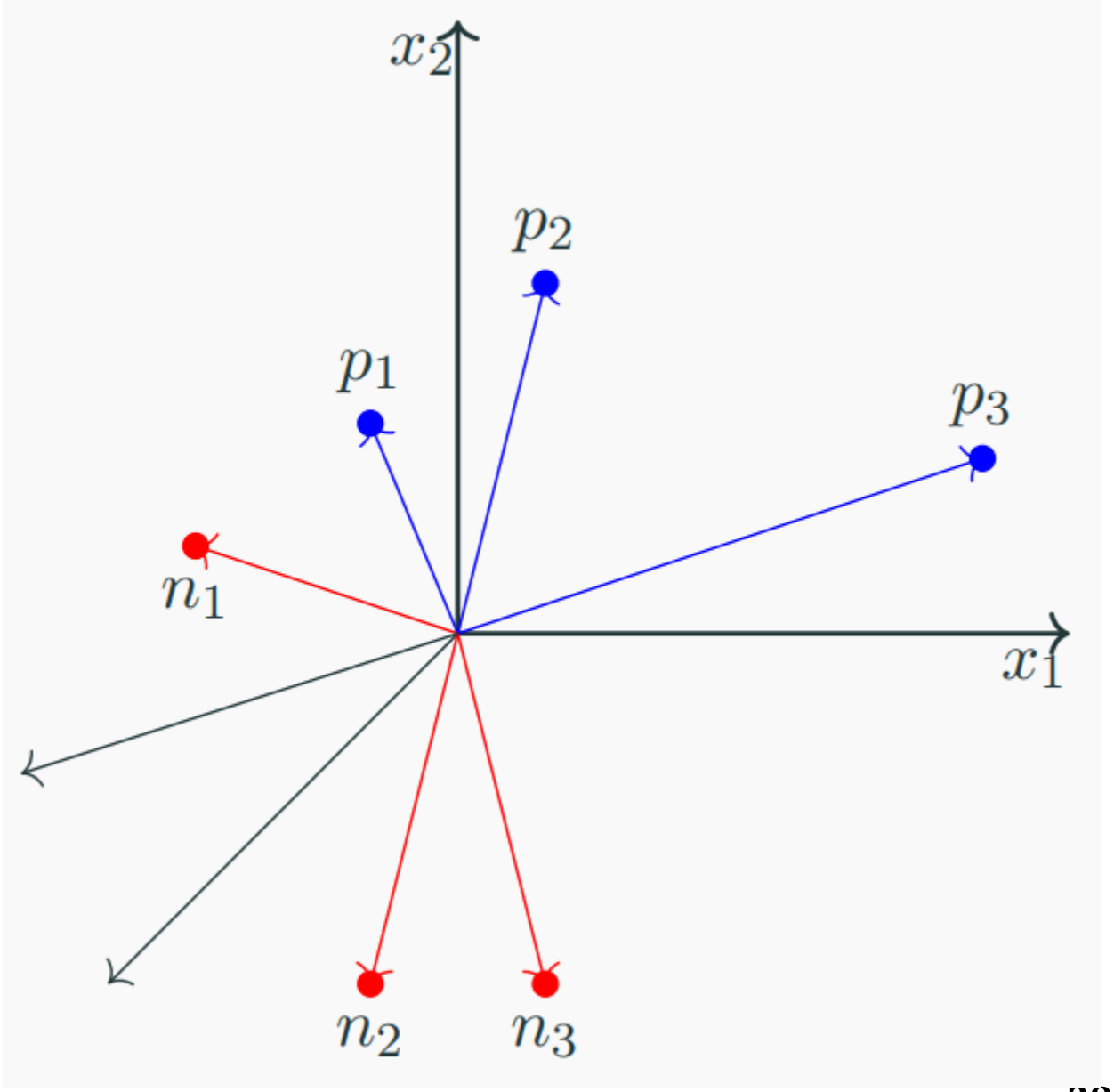
We now run the algorithm by randomly going over the points

Decision boundary – Update w.r.t. p1

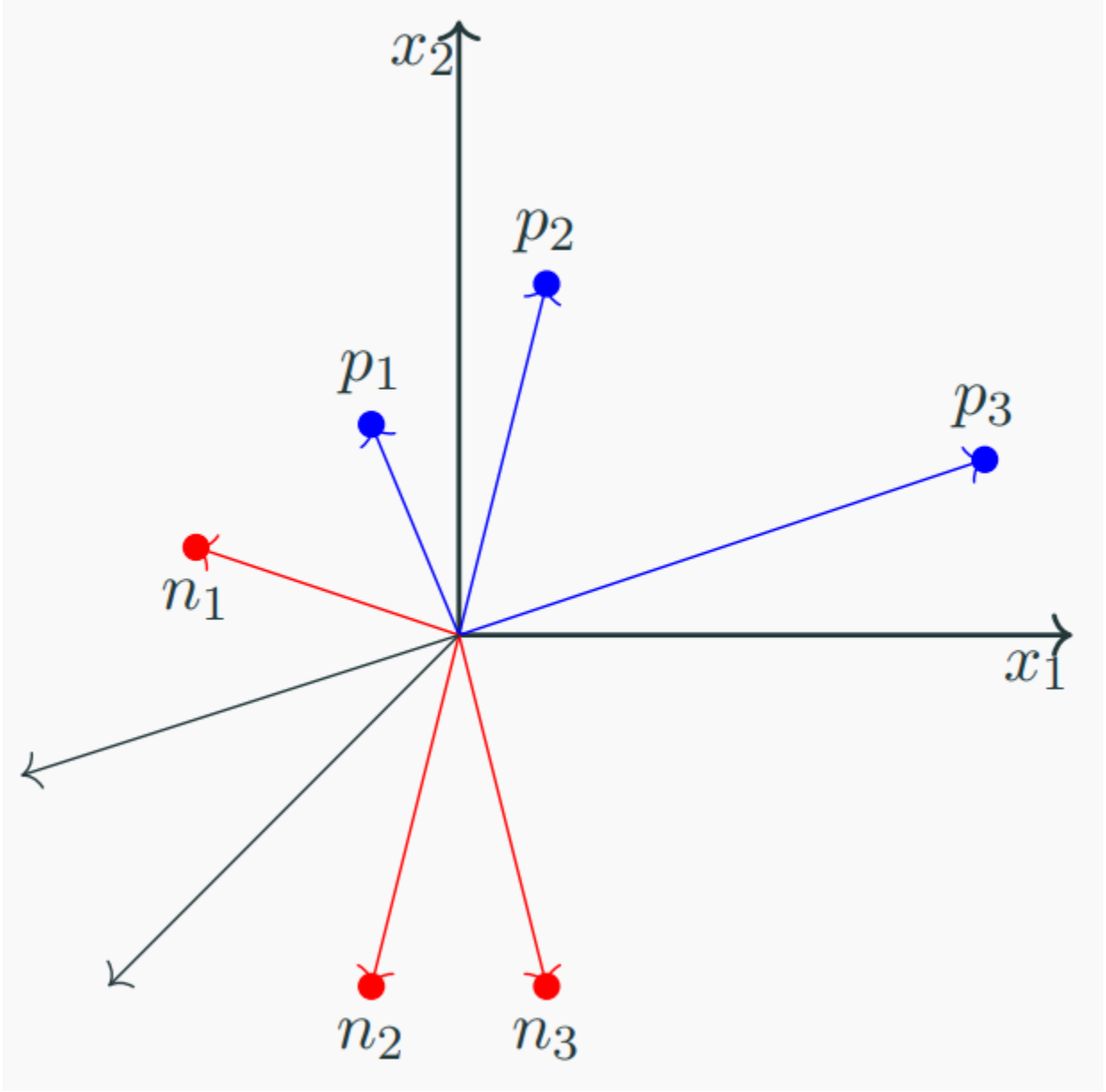


Decision boundary – Update w.r.t. p1

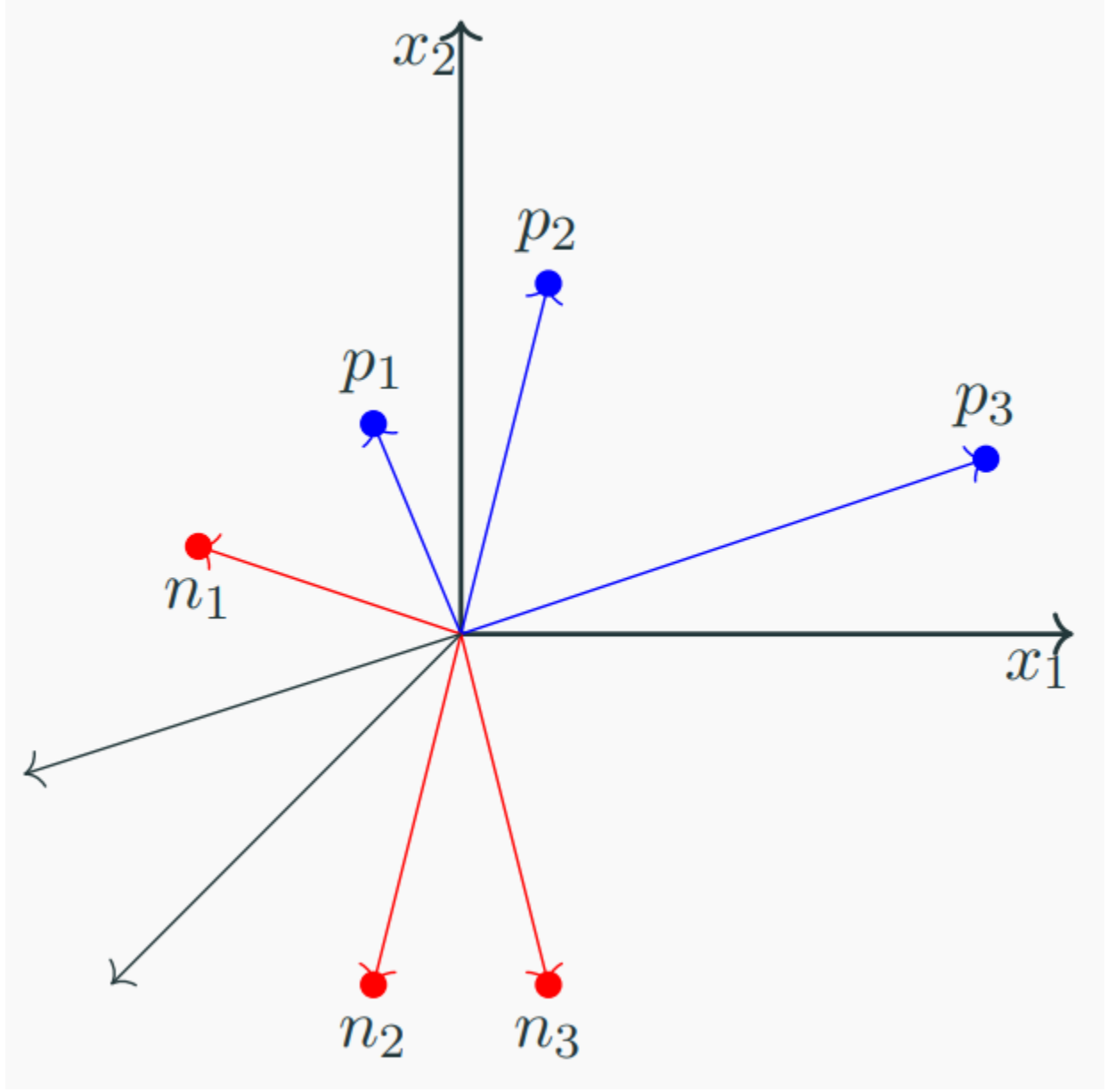


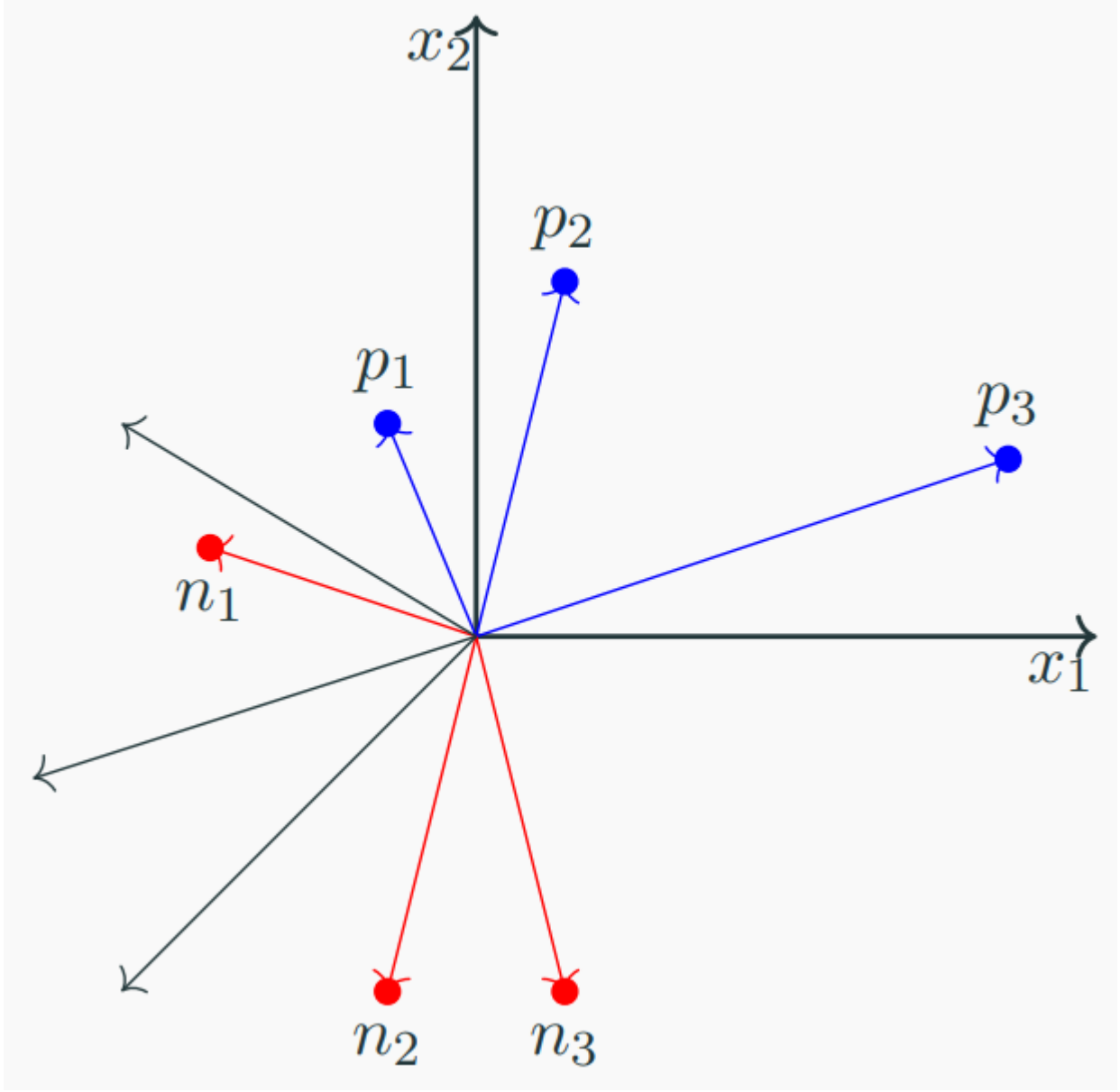


Decision boundary – Update w.r.t. p2

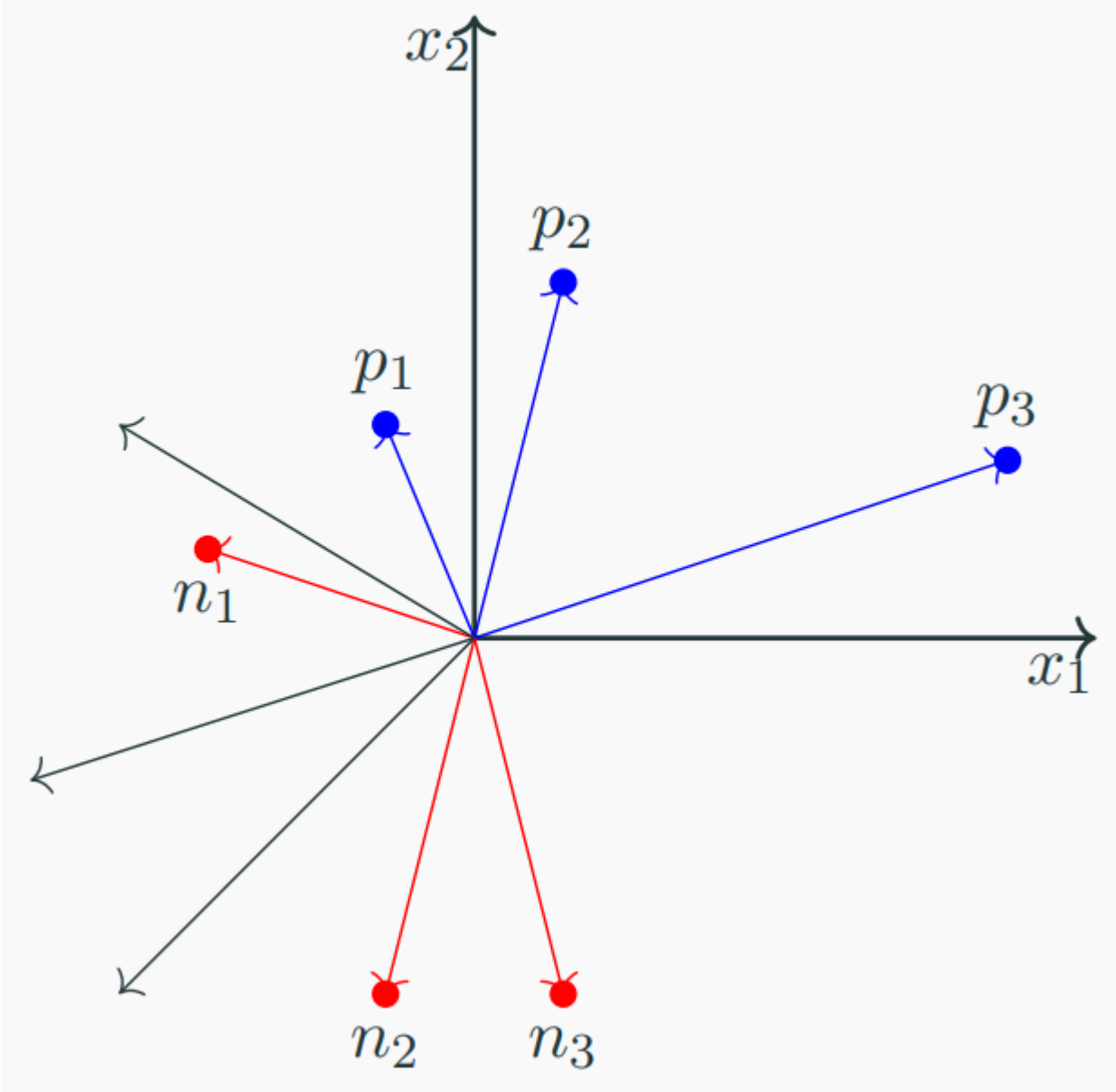


Decision boundary – Update w.r.t. p2

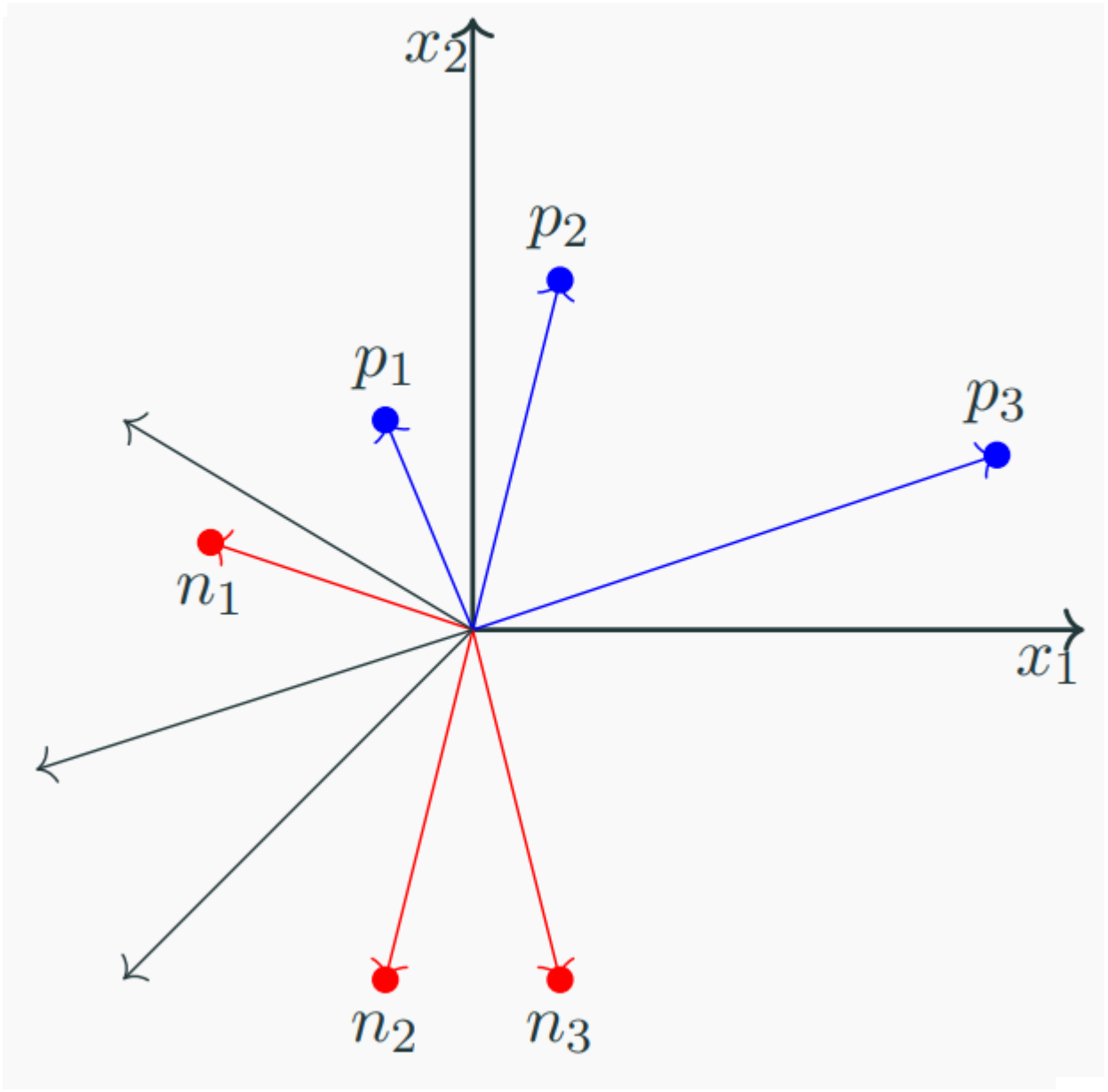


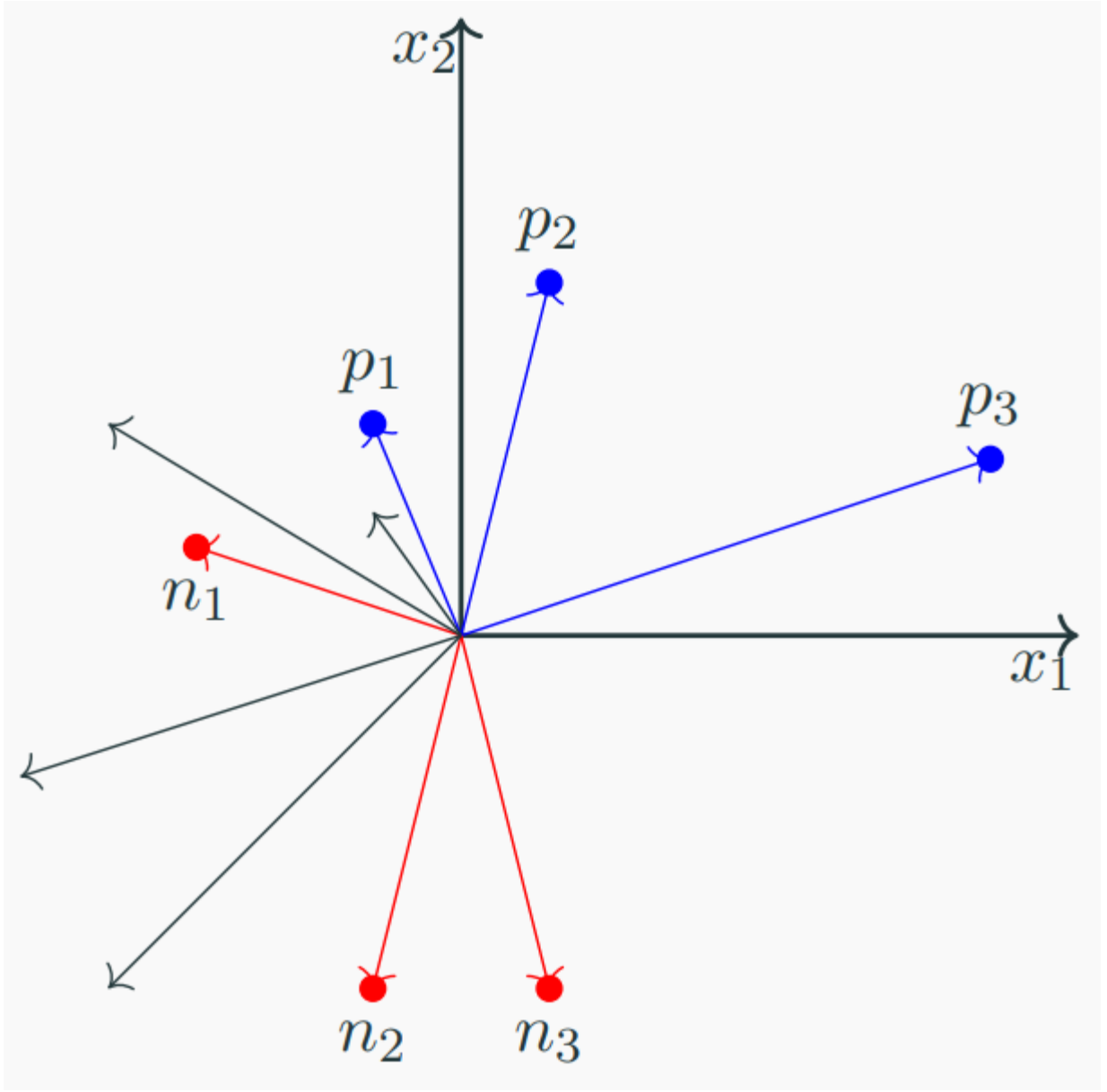


Decision boundary – Update w.r.t. n1

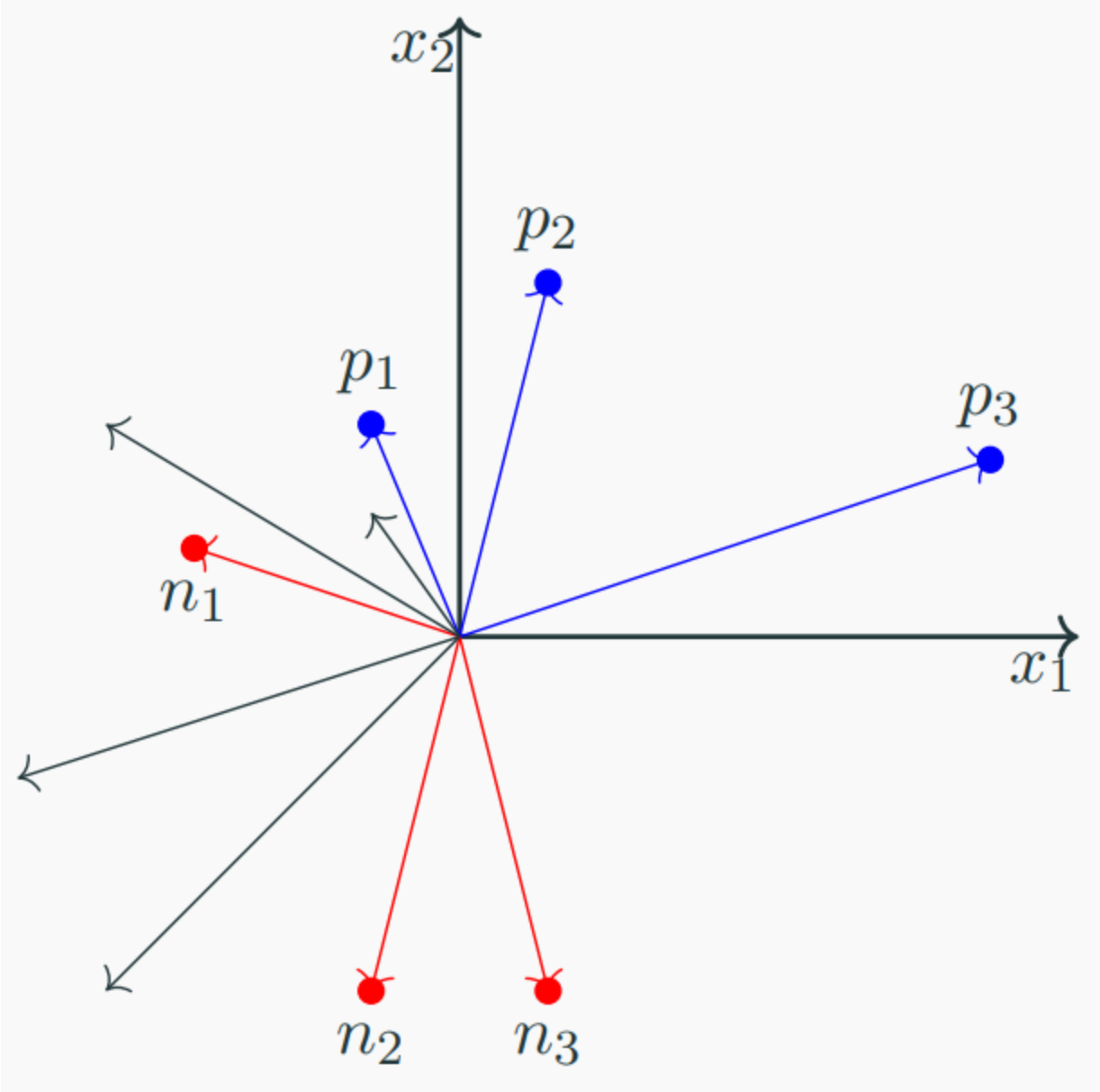


Decision boundary – Update w.r.t. n1

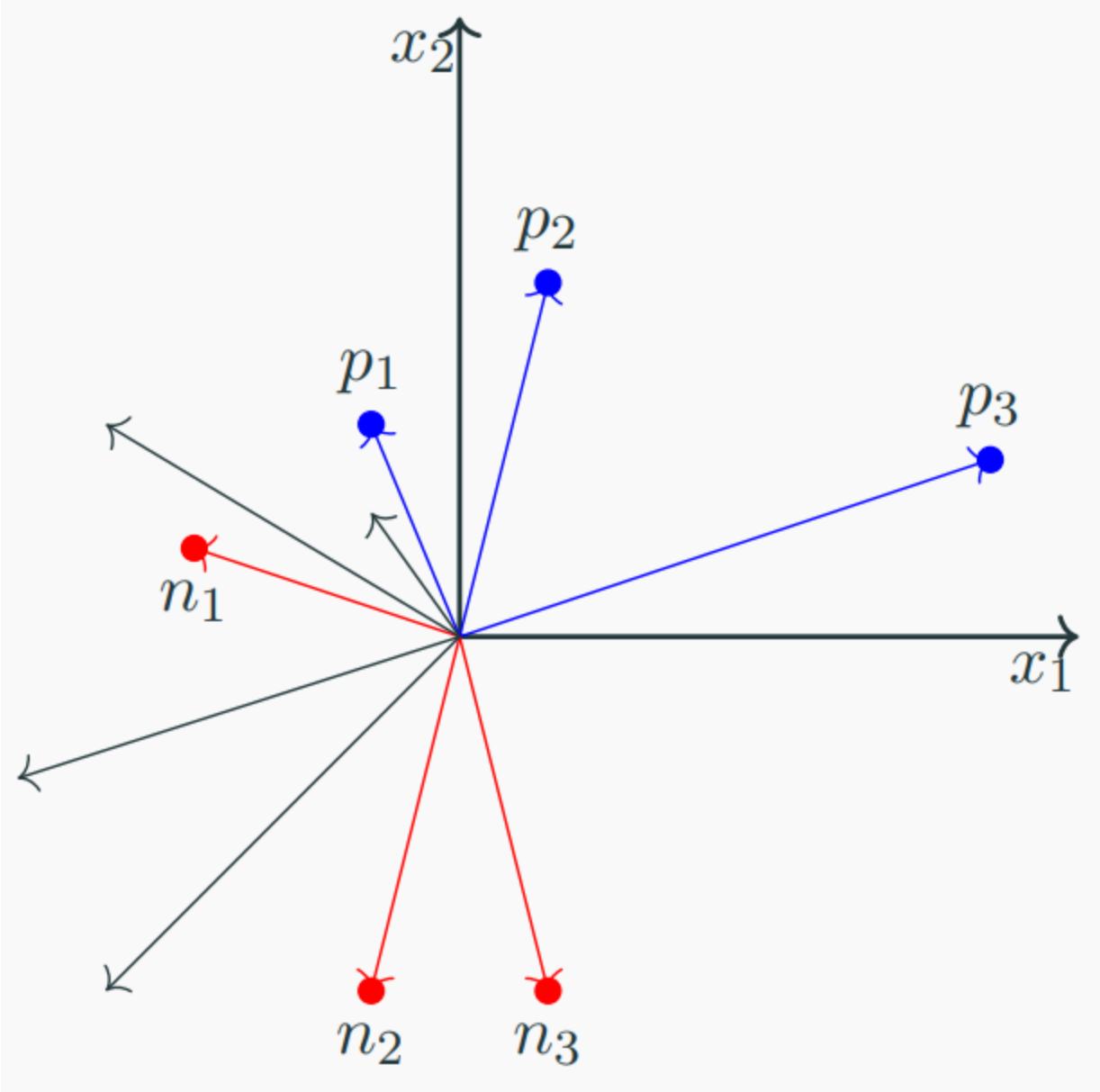




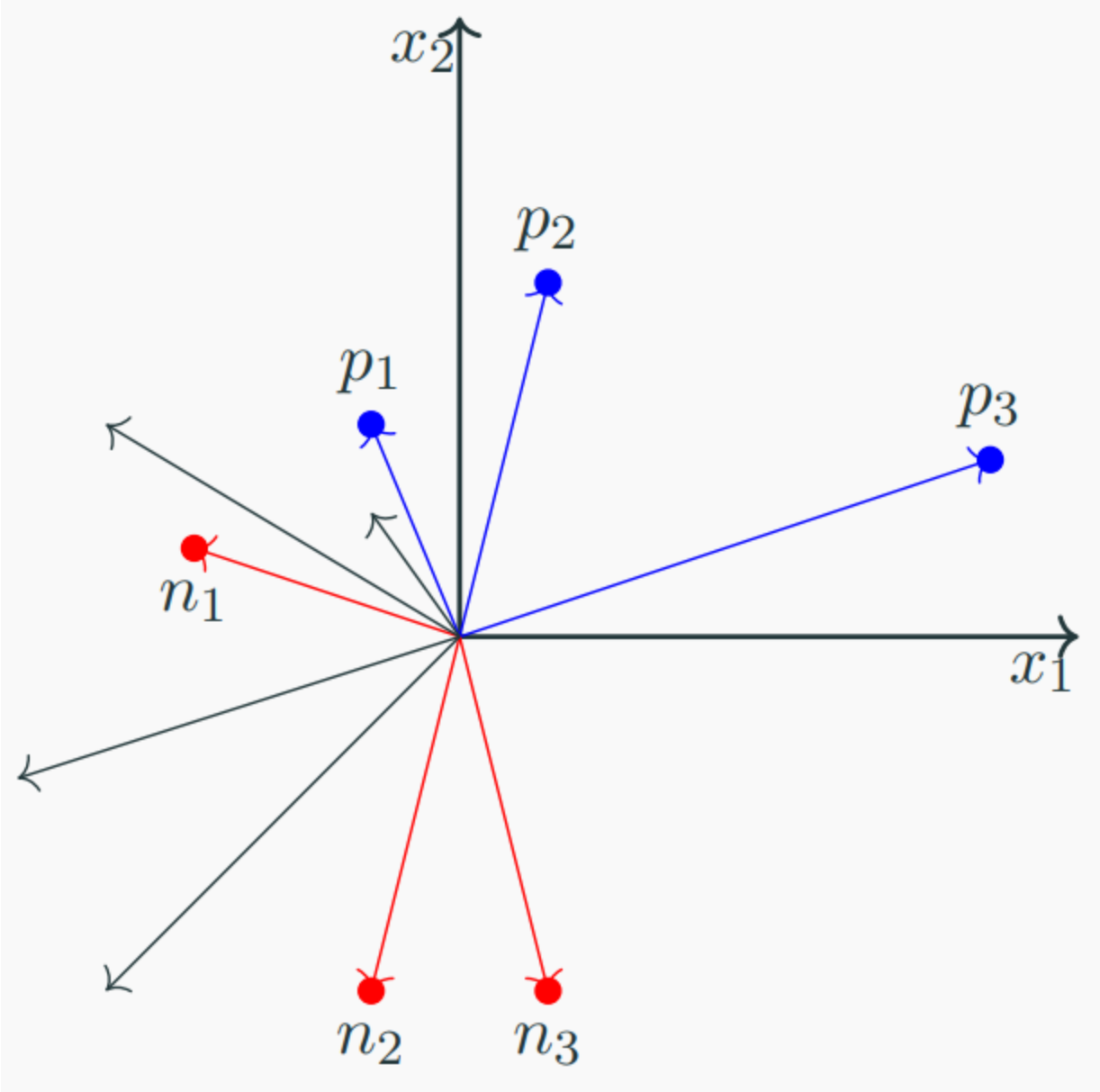
Decision boundary – Update w.r.t. n2

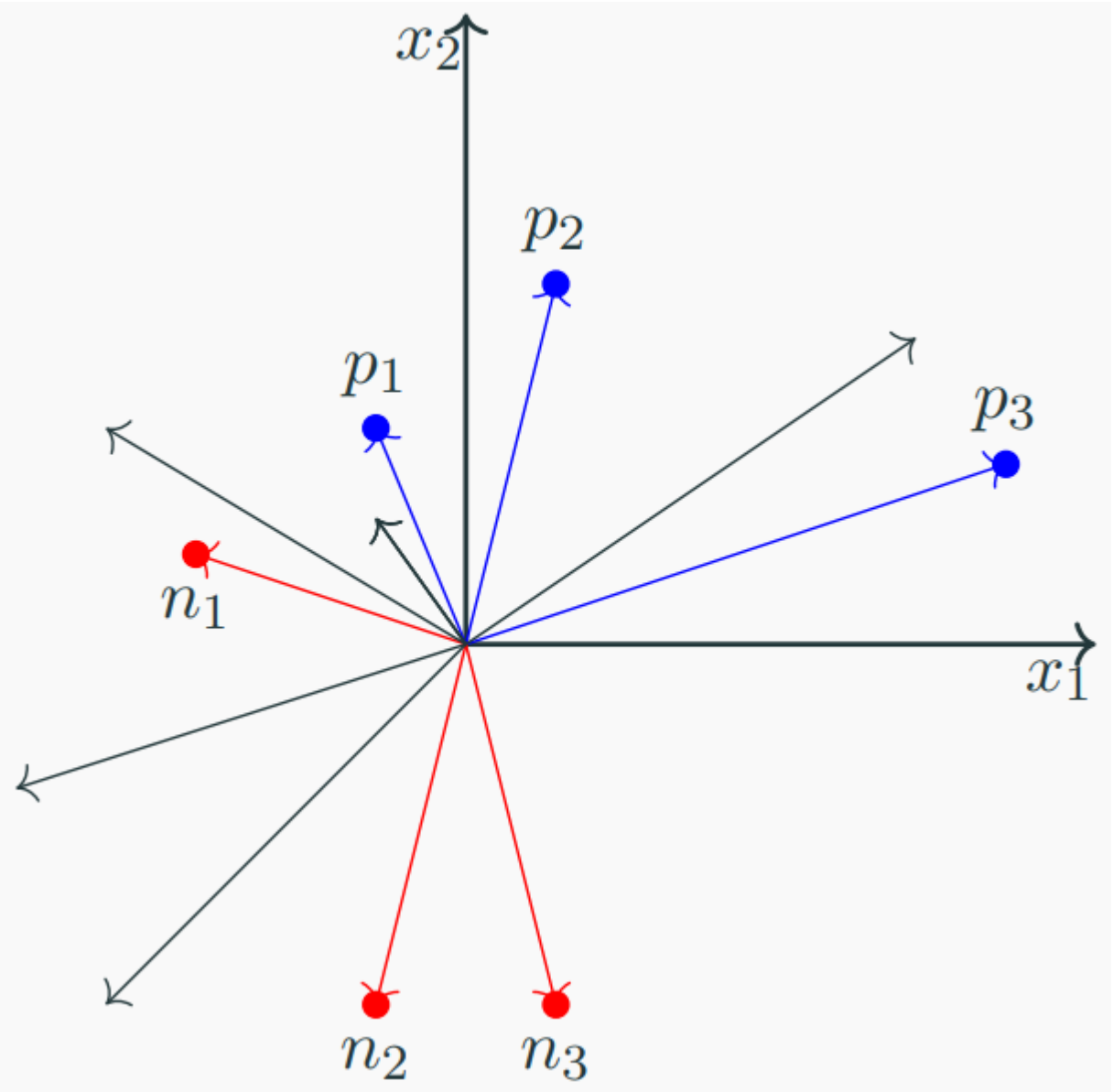


Decision boundary – Update w.r.t. p3

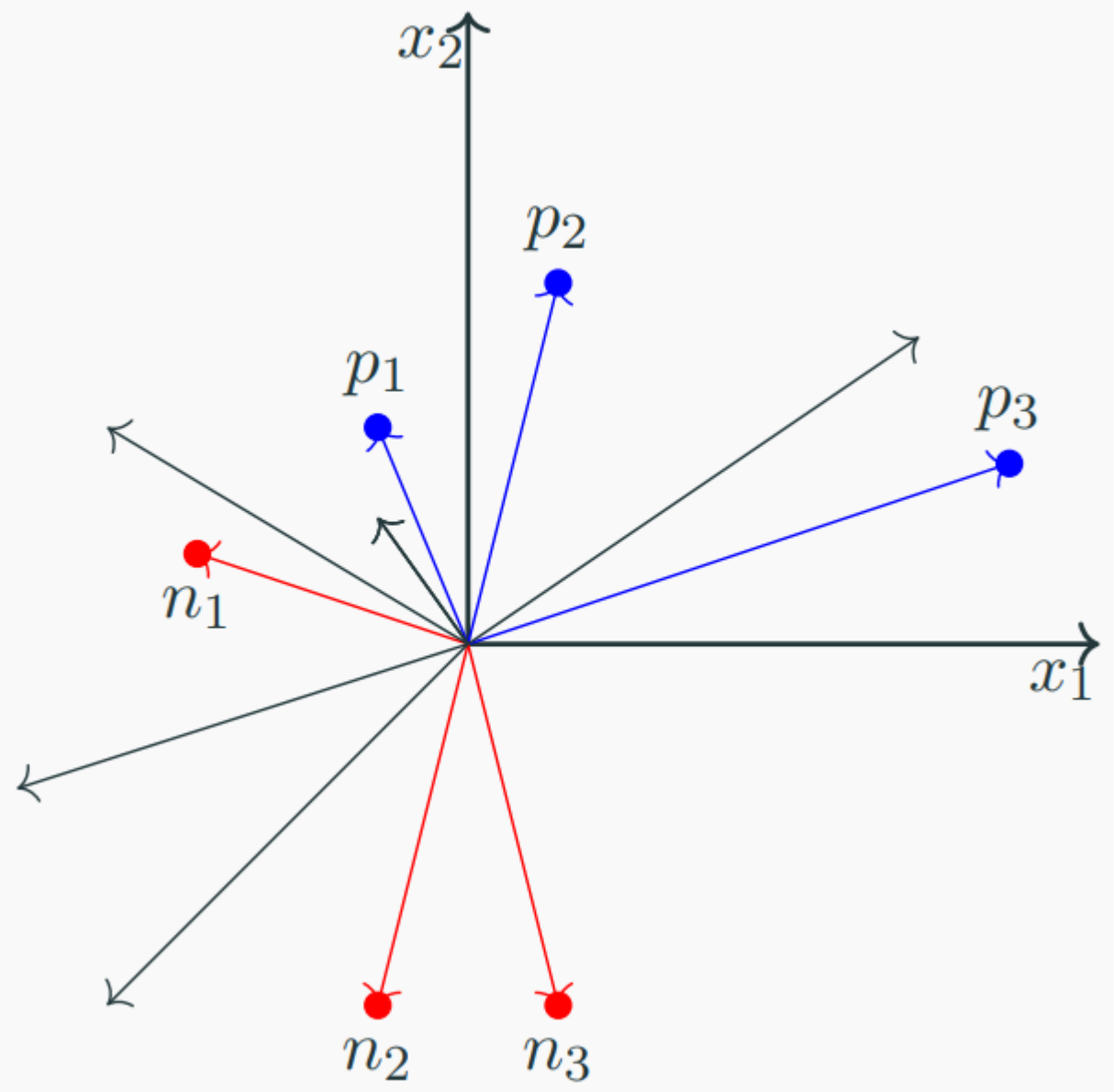


Decision boundary – Update w.r.t. p3





Decision boundary – After 5 update steps



Are further updates needed?

Coding example

Review of key concepts

1. Does PLA converge to a unique decision boundary?

Review of key concepts

1. Does PLA converge to a unique decision boundary?

No, it converges to ANY line (and not the BEST line) that perfectly classifies the dataset.

Final decision boundary will depend on:

- Initial weights
- Order of data

Review of key concepts

2. Can PLA classify data that is not linearly separable?

Review of key concepts

2. Can PLA classify data that is not linearly separable?

No, because a decision boundary that PERFECTLY classifies such a dataset does not exist. There will be always be a few misclassifications.

PLA will loop indefinitely.

Thank you for your time!